

`torch.Tensor.scatter_`

The `scatter_` method in PyTorch is used to **write values from a source tensor into a target tensor at specified indices**. It is an **in-place operation** (indicated by the trailing underscore `_`), meaning it modifies the tensor it is called on.

This method is particularly useful when you need to update specific elements of a tensor based on indices.

Syntax:

python

Copy

```
tensor.scatter_(dim, index, src)
```

Parameters:

dim : The dimension along which to index. For example, `dim=0` means updating rows, and `dim=1` means updating columns.

index : A tensor containing the indices of the elements to update. The shape of `index` must match the shape of `src`.

src : The source tensor containing the values to scatter into the target tensor. It can also be a scalar value.

How it works:

For each element in `src`, the `scatter_` method writes it into the target tensor at the position specified by the corresponding index in the `index` tensor.

The `dim` parameter determines the axis along which the indices are applied.

Example 1: Basic Usage

python

Copy

```
import torch
```

```
# Create a target tensor
```

```
target = torch.zeros(3, 5) # 3x5 tensor filled with zeros
```

```
# Indices where values will be written
```

```
indices = torch.tensor([[0, 1, 2, 0, 0], [2, 0, 0, 1, 2]])
```

```
# Source tensor containing values to scatter
```

```
src = torch.tensor([[1, 2, 3, 4, 5], [6, 7, 8, 9, 10]])
```

```
# Scatter values from src into target at the specified indices along dim=0
```

```
target.scatter_(0, indices, src)
```

```
print(target)
```

Output:

Copy

```
tensor([[1, 7, 8, 4, 5],
        [0, 2, 0, 9, 0],
        [6, 0, 3, 0, 10]])
```

Explanation:

The `indices` tensor specifies where to place the values from `src` into `target` along `dim=0`.

For example:

`src[0, 0] = 1` is placed at `target[indices[0, 0], 0] = target[0, 0]` .

`src[1, 0] = 6` is placed at `target[indices[1, 0], 0] = target[2, 0]` .

Example 2: Scalar Source

You can also use a scalar value as the `src` parameter. In this case, the same value will be scattered into the target tensor at the specified indices.

```
python
import torch
```

Copy

```
# Create a target tensor
```

```
target = torch.zeros(3, 5)
```

```
# Indices where values will be written
```

```
indices = torch.tensor([[0, 1, 2, 0, 0], [2, 0, 0, 1, 2]])
```

```
# Scatter the value 99 into target at the specified indices along dim=0
```

```
target.scatter_(0, indices, 99)
```

```
print(target)
```

Output:

Copy

```
tensor([[99, 99, 99, 99, 99],
        [ 0, 99,  0, 99,  0],
        [99,  0, 99,  0, 99]])
```

Example 3: Scattering Along `dim=1`

You can scatter values along a different dimension by changing the `dim` parameter.

```
python
import torch
```

Copy

```
# Create a target tensor
```

```
target = torch.zeros(3, 5)
```

```
# Indices where values will be written
```

```
indices = torch.tensor([[0, 1, 2, 0, 0], [2, 0, 0, 1, 2]])
```

```
# Source tensor containing values to scatter
```

```
src = torch.tensor([[1, 2, 3, 4, 5], [6, 7, 8, 9, 10]])
```

```
# Scatter values from src into target at the specified indices along dim=1
```

```
target.scatter_(1, indices, src)
```

```
print(target)
```

Output:

Copy

```
tensor([[1, 2, 3, 0, 0],
        [6, 7, 8, 0, 0],
        [0, 0, 0, 0, 0]])
```

[0, 0, 0, 0, 0]))

Explanation:

Here, the values are scattered along `dim=1` (columns).

For example:

`src[0, 0] = 1` is placed at `target[0, indices[0, 0]] = target[0, 0]` .

`src[1, 0] = 6` is placed at `target[1, indices[1, 0]] = target[1, 2]` .

Key Notes:

- . **In-Place Operation:** The `scatter_` method modifies the tensor in place. If you don't want to modify the original tensor, use `torch.scatter` instead (without the underscore).
- . **Shape Requirements:** The shapes of `index` and `src` must match, and they must be compatible with the target tensor's shape along the specified dimension.
- . **Use Cases:** This method is often used in advanced operations like one-hot encoding, sparse updates, or custom indexing.